ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.

Assignment : Evaluating Virtual Memory Strategies for Diverse Computing Environments
Operating Systems : CSE 4501

Student ID: 210042112

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

April 29, 2025

Abstract

Virtual memory is essential for multitasking and effective memory utilization. In this assignment, analyze how different virtual memory techniques are suited for different computing environments (e.g., desktop, mobile, embedded systems, cloud servers).

Contents

1 Desktop Systems **4**
- 1.1 Requirements . . . 4
- 1.2 Constraints . . . 4
- 1.3 Proposed Virtual Memory Strategy . . . 4
- 1.4 Comparison with Current Systems . . . 4
- 1.5 Justification . . . 4

2 Mobile Devices **5**
- 2.1 Requirements . . . 5
- 2.2 Constraints . . . 5
- 2.3 Proposed Virtual Memory Strategy . . . 5
- 2.4 Comparison with Current Systems . . . 5
- 2.5 Justification: . . . 5

3 Embedded Systems **6**
- 3.1 Requirements . . . 6
- 3.2 Constraints . . . 6
- 3.3 Proposed Virtual Memory Strategy . . . 6
- 3.4 Comparison with Current Systems . . . 6
- 3.5 Justification . . . 6

1 Desktop Systems

1.1 Requirements

Desktop systems, such as personal computers, support diverse workloads (e.g., gaming, productivity software, development tools) requiring high performance, multitasking, and large memory availability. They typically have ample physical memory (8–64 GB) and powerful CPUs.

1.2 Constraints

Users expect responsiveness despite running multiple applications. Memory-intensive tasks (e.g., video editing) demand efficient memory allocation. Cost is less restrictive compared to mobile or embedded systems.

1.3 Proposed Virtual Memory Strategy

- **Technique:** Demand Paging with a Least Recently Used (LRU) Page Replacement Algorithm
  - **Paging:** Use fixed-size pages (e.g., 4 KB) to simplify memory management and enable efficient swapping. Demand paging loads pages only when accessed, reducing initial memory overhead.
  - **Page Replacement:** LRU approximates optimal replacement by tracking page access history, ideal for workloads with temporal locality (e.g., frequently accessed applications). Use a second-chance algorithm to approximate LRU efficiently, reducing overhead.
  - **Additional Features:** Implement a working set model to ensure active processes have sufficient pages in memory, minimizing thrashing. Enable memory compression to store less frequently used pages in compressed form, as seen in modern systems like Windows.
- **Rationale:**Desktop workloads exhibit high locality and require fast context switching. Demand paging balances memory usage, while LRU ensures frequently used pages remain in memory. The working set model prevents thrashing during multitasking

1.4 Comparison with Current Systems

- **Typical Implementation:** Modern desktop OSes like Windows 11 and Linux use demand paging with LRU-based or clock-based (second-chance) page replacement. Linux employs a variant of the working set model and supports memory compression (zswap). Windows uses SuperFetch to prefetch pages based on usage patterns.
- **Alignment:** The proposed strategy aligns closely with current implementations. The use of LRU and memory compression mirrors Linux's approach, while the working set model reflects Windows' process prioritization. My strategy emphasizes LRU over clock algorithms for better accuracy in high-memory systems, which is practical given desktop hardware capabilities.

1.5 Justification

- **Theoretical Soundness:** LRU leverages temporal locality, a principle desktops exploit due to repeated application use. Demand paging minimizes I/O overhead, and the working set model ensures performance stability.
- **Practical Effectiveness:** Desktop systems have sufficient CPU and disk resources to handle LRU's bookkeeping. Memory compression reduces swap usage, improving responsiveness, as evidenced by Linux's ***zswap*** performance gains.

2 Mobile Devices

2.1 Requirements

Mobile devices (e.g., smartphones, tablets) prioritize energy efficiency, responsiveness, and smooth user interfaces. They run multiple apps (e.g., social media, games) with moderate memory (4–12 GB) and constrained battery life.

2.2 Constraints

Limited battery capacity demands low power consumption. Memory is shared among apps and the OS, requiring aggressive resource management. Storage is often flash-based, with slower write speeds than desktop SSDs.

2.3 Proposed Virtual Memory Strategy

- **Technique:** Paging with Low-Memory Killer (LMK) and ZRAM
  - **Paging:** Use demand paging with small page sizes (e.g., 4 KB) to support fine-grained memory allocation. Avoid segmentation due to its complexity and overhead.
  - **Page Replacement:** Instead of traditional algorithms like LRU, use a Low-Memory Killer mechanism to terminate less critical processes when memory is low, preserving foreground app performance. Prioritize pages based on app importance (e.g., foreground vs. background).
  - **ZRAM:** Implement ZRAM, a compressed swap space in RAM, to store pages in memory rather than writing to flash storage, reducing power consumption and wear.
- **Rationale:**Mobile devices prioritize foreground apps and battery life. LMK ensures critical apps remain responsive by reclaiming memory from background processes. ZRAM reduces flash I/O, saving energy and extending storage lifespan.

2.4 Comparison with Current Systems

- **Typical Implementation:** Android, the dominant mobile OS, uses demand paging, LMK, and ZRAM extensively. iOS employs a similar approach, prioritizing foreground apps and using memory compression. Android's LMK categorizes processes (e.g., cached, background) and kills them based on memory pressure.
- **Alignment:** The proposed strategy mirrors Android's approach, particularly in using LMK and ZRAM. My emphasis on avoiding traditional page replacement algorithms reflects Android's design, which prioritizes process termination over page swapping to maintain UI fluidity.

2.5 Justification:

- **Theoretical Soundness:** ZRAM leverages compression to reduce I/O, aligning with virtual memory's goal of efficient resource use. LMK is an application of memory management tailored to mobile constraints, ensuring responsiveness by sacrificing less critical processes.
- **Practical Effectiveness:** ZRAM reduces power consumption, critical for battery-limited devices, as shown by Android's adoption. LMK's process termination is faster than swapping on flash storage, maintaining performance under memory pressure.

3 Embedded Systems

3.1 Requirements

Embedded systems (e.g., IoT devices, automotive controllers) prioritize reliability, low latency, and minimal resource usage. They often have limited memory (e.g., 512 KB–2 MB) and run real-time tasks.

3.2 Constraints

Hardware is resource-constrained, with no swap space or secondary storage in many cases. Real-time constraints demand predictable performance, and power consumption is critical in battery-powered devices.

3.3 Proposed Virtual Memory Strategy

- **Technique:**Static Paging or No Virtual Memory
  - **Static Paging:** For systems with minimal memory, use simple paging without demand paging. Preload all required pages at startup to avoid runtime swapping. Use fixed-size pages (e.g., 1 KB) to minimize fragmentation.
  - **No Virtual Memory:** For hard real-time systems, disable virtual memory entirely to eliminate overhead from page faults and address translation. Use flat memory addressing with direct physical memory access.
  - **Page Replacement:** If paging is used, employ a simple First-In-First-Out (FIFO) algorithm to minimize computational overhead, as LRU or clock algorithms are too resource-intensive.
- **Rationale:**Embedded systems prioritize predictability and efficiency. Static paging eliminates runtime overhead, while disabling virtual memory ensures deterministic performance for real-time tasks.

3.4 Comparison with Current Systems

- **Typical Implementation:** Many embedded systems, especially real-time ones (e.g., RTOS like FreeRTOS), avoid virtual memory to ensure predictability. Systems with virtual memory, like Linux-based IoT devices, use simple paging with minimal swapping. FIFO or no replacement is common due to resource constraints.
- **Alignment:** The proposed strategy aligns with RTOS practices by prioritizing no virtual memory for hard real-time systems. Static paging is used in Linux-based embedded systems, though my approach avoids demand paging to reduce complexity, which is practical for constrained hardware.

3.5 Justification

- **Theoretical Soundness:** Disabling virtual memory eliminates address translation overhead, critical for real-time constraints. Static paging simplifies memory management, reducing fragmentation in low-memory systems.
- **Practical Effectiveness:** Static paging and FIFO require minimal CPU resources, fitting embedded systems' constraints. Avoiding virtual memory ensures predictable latency, as seen in automotive controllers using RTOS.

The proposed virtual memory strategies—demand paging with LRU for desktops, paging with LMK and ZRAM for mobile devices, and static paging or no virtual memory for embedded systems—are tailored to each environment's unique requirements. These strategies align with current implementations while emphasizing theoretical principles like locality, efficiency, and predictability. By addressing performance, power, and resource constraints, the strategies ensure both theoretical soundness and practical effectiveness.